

Laboratorio 01 · Entorno Docker y primer mongod

Duración: 25 minutos · Carpeta de trabajo: `dia1/`

Objetivos

Al terminar este laboratorio habrás arrancado un `mongod` standalone a partir de un fichero de configuración, te habrás conectado a él de dos formas distintas, habrás comprobado con qué opciones se está ejecutando y habrás verificado que los datos sobreviven a un reinicio del contenedor.

Contexto

En producción MongoDB se instala normalmente con los paquetes oficiales (`apt`, `yum` / `dnf`) y se gestiona como un servicio de `systemd`. En el curso usamos la imagen oficial `mongodb/mongodb-community-server`, que contiene exactamente los mismos binarios. Lo que cambia es cómo arrancamos y paramos el proceso: en vez de `systemctl start mongod` usaremos `docker compose up`.

Paso 1 · Arrancar el servidor

```
cd dia1
docker compose up -d solo toolbox
docker compose ps
```

Mira el log de arranque del servidor. Desde MongoDB 4.4 el log es **JSON estructurado**, una línea por evento:

```
docker compose logs solo | head -n 40
```

Busca en la salida las líneas con `"msg":"Build Info"`, `"msg":"Options set by command line"` y `"msg":"Waiting for connections"`.

Paso 2 · Conectarse con mongosh

Primera forma: ejecutando `mongosh` dentro del propio contenedor del servidor, como harías con SSH en un servidor real:

```
docker compose exec solo mongosh
```

```
> db.version()
> db.hello()
> exit
```

Segunda forma: desde el **toolbox**, que es otra máquina de la misma red. Esta es la forma en la que trabajaremos casi siempre:

```
docker compose exec toolbox bash
mongosh "mongodb://solo:27017"
```

Paso 3 · ¿Con qué configuración se está ejecutando?

Dentro de `mongosh`:

```
> db.serverCmdLineOpts()
> db.serverBuildInfo().version
> db.hostInfo().system
> db.serverStatus().storageEngine
```

1. ¿Qué valor tiene `cacheSizeGB` según `serverCmdLineOpts()`? ¿De dónde sale?
2. ¿Cuántos núcleos y cuánta memoria ve MongoDB según `hostInfo()`?

▼ Ver solución

`serverCmdLineOpts().parsed` muestra la configuración ya interpretada, combinando el fichero `/etc/mongod.conf` y los parámetros de línea de comandos. `cacheSizeGB` vale `0.5` porque lo fijamos en `dia1/config/mongod.conf`. En `hostInfo().system` verás `numCores` y `memSizeMB`; si Docker tiene un límite de memoria, `memLimitMB` refleja ese límite, que es el que MongoDB usa para calcular la caché por defecto.

Paso 4 · Crear datos y comprobar la persistencia

```
> use tienda
> db.clientes.insertOne({ nombre: "Ana", ciudad: "Madrid", alta: new
Date() })
```

```
> db.clientes.find()
> show dbs
> exit
```

Sal del toolbox (`exit`) y reinicia el servidor desde tu equipo:

```
docker compose restart solo
docker compose logs solo | grep -i "shutdown"
```

Vuelve a conectarte y comprueba que el documento sigue ahí. Los datos viven en el **volumen** `solo-data`, no en el contenedor:

```
docker volume ls | grep dia1
```

Paso 5 · Herramientas disponibles

Comprueba qué herramientas tienes en el toolbox:

```
docker compose exec toolbox bash -c "mongosh --version; mongodump --
version | head -1; mongostat --version | head -1"
```

Para terminar

Deja `solo` y `toolbox` en marcha: los usaremos en los laboratorios 03 y 04.

Preguntas de repaso

1. ¿Qué diferencia hay entre parar el contenedor con `docker compose stop` y con `docker kill`? ¿Por qué `stop_grace_period` está a 60 segundos en el `compose.yml`?
2. Si borras el contenedor con `docker compose rm`, ¿pierdes los datos? ¿Y con `docker compose down -v`?

▼ Ver solución

`stop` envía `SIGTERM` y `mongod` hace un apagado limpio: termina las operaciones en curso, hace un checkpoint de WiredTiger y cierra los ficheros. `kill` envía `SIGKILL` y el proceso muere al instante; al arrancar, WiredTiger tendrá que recuperarse desde el último checkpoint más el journal. Damos 60 segundos porque en instancias grandes el apagado

limpio puede tardar más de los 10 segundos por defecto de Docker. `docker compose rm` borra el contenedor pero no el volumen; `down -v` borra también los volúmenes y, con ellos, los datos.